

# Expo Setup Guide

Installing React Native (via Expo) and creating your first app — Windows

## Contents

1. Install prerequisites
2. Create your first app
3. Run it on your phone (Expo Go)
4. Project structure
5. Everyday commands
6. Optional: Android emulator setup
7. Troubleshooting

**Why Expo, and not the bare React Native CLI?** Expo gives you a working app in minutes with no Android Studio or native build tools required to get started — you write JavaScript/TypeScript, and the **Expo Go** app on your own phone runs it instantly over Wi-Fi. You can still eventually build a real native app (APK) from the same project later; nothing about starting with Expo locks you out of that.

## 1. Install prerequisites

### Node.js (required)

Expo's tooling runs on Node. Install the current **LTS** version.

1. Go to `nodejs.org` and download the "LTS" Windows installer (.msi).
2. Run the installer, accepting the defaults (this also installs `npm`, Node's package manager).
3. Open a new PowerShell window and confirm it worked:

```
node -v
npm -v
```

Both commands should print a version number. If you get "not recognized", close and reopen PowerShell (the installer updates PATH, but only for new terminal windows) — if it still fails, restart your PC.

### Git (required)

1. Download from `git-scm.com`, run the installer with default options.
2. Verify:

```
git --version
```

### A code editor (recommended: VS Code)

1. Download from `code.visualstudio.com` and install.
2. Recommended extensions (Extensions panel, `Ctrl+Shift+X`):
  - **ESLint** — catches JS/TS mistakes as you type
  - **Prettier** — auto-formats code on save
  - **React Native Tools** — debugging support inside VS Code

### The Expo Go app (on your phone)

This is how you'll actually see the app running, without installing anything else on Windows.

- Android phone: install **Expo Go** from the Play Store.
- iPhone: install **Expo Go** from the App Store (fine to use even though you're developing on Windows).

Your phone and your Windows PC must be on the **same Wi-Fi network** for this to work — that's the single most common thing that trips people up (see Troubleshooting).

## 2. Create your first app

You don't install a global "Expo CLI" package — `npx` downloads and runs the latest version fresh each time, so you're never stuck on an outdated tool.

```
# Pick a folder to work in first, e.g.:  
cd Documents  
npx create-expo-app@latest my-first-app  
cd my-first-app
```

This scaffolds a ready-to-run project with example screens already wired up using **Expo Router** (file-based navigation — each file under `app/` becomes a screen).

## 3. Run it on your phone (Expo Go)

```
npx expo start
```

1. A QR code appears in the terminal (and a browser tab opens with the same QR code, bigger).
2. Open the **Expo Go** app on your phone.
3. Android: tap "Scan QR code" inside Expo Go. iPhone: just use the regular Camera app and tap the notification.
4. The app builds a JavaScript bundle and loads on your phone — first load takes a few seconds.

Any time you save a file, your phone updates almost instantly ("Fast Refresh") — no rebuild, no reinstalling. This is the fast feedback loop that makes Expo good for learning.

## 4. Project structure (what you're looking at)

| Path                         | What it is                                                          |
|------------------------------|---------------------------------------------------------------------|
| <code>app/</code>            | Your screens. <code>app/index.tsx</code> is the first screen shown. |
| <code>app/_layout.tsx</code> | Wraps every screen — shared navigation/providers go here.           |
| <code>assets/</code>         | Images, fonts, icons.                                               |
| <code>package.json</code>    | Project dependencies and scripts ( <code>npm run ...</code> ).      |
| <code>app.json</code>        | App name, icon, splash screen, and other Expo config.               |
| <code>node_modules/</code>   | Downloaded dependencies — never edit, never commit to git.          |

## 5. Everyday commands

| What you want to do                                               | Command                                    |
|-------------------------------------------------------------------|--------------------------------------------|
| Start the dev server                                              | <code>npx expo start</code>                |
| Start and clear the bundler cache (fixes weird stale-code issues) | <code>npx expo start -c</code>             |
| Add a package                                                     | <code>npx expo install some-package</code> |
| Type-check the project (if using TypeScript)                      | <code>npx tsc --noEmit</code>              |
| See installed Expo SDK / tool versions                            | <code>npx expo --version</code>            |

Prefer `npx expo install some-package` over plain `npm install` for anything Expo-related — it picks the exact version that matches your Expo SDK, which `npm install` alone doesn't know to do.

## 6. Optional: Android emulator (skip this at first)

Expo Go on your own phone is the fastest way to get started and is enough for a while. Only set this up once you specifically need to test without a physical phone, or need a native module Expo Go doesn't support.

1. Install **Android Studio** from `developer.android.com/studio`.
2. During setup, let it install the Android SDK and an emulator image (the default "More Options → SDK Manager" flow is fine).
3. Open Android Studio → **More Actions** → **Virtual Device Manager** → create a device (e.g. Pixel 8) and let it download a system image.
4. Add these to your Windows environment variables (System Properties → Environment Variables):
  - `ANDROID_HOME = C:\Users\<you>\AppData\Local\Android\Sdk`
  - Add `%ANDROID_HOME%\platform-tools` and `%ANDROID_HOME%\emulator` to `Path`
5. Boot the emulator from Android Studio's Device Manager, then, from the project folder:

```
npx expo run:android
```

First run does a full native build (several minutes, downloads Gradle dependencies) and generates an `android/` folder in your project. After that, JS changes hot-reload the same way as Expo Go.

## 7. Troubleshooting

### QR code won't connect / phone can't reach the dev server

- Confirm the phone and PC are on the **same Wi-Fi network** (not phone on mobile data).
- Some routers/guest networks isolate devices from each other — try a personal hotspot from the PC if you're stuck on a locked-down network (school/office Wi-Fi is a common culprit).
- Windows Firewall may be blocking Node — if a "Windows Defender Firewall has blocked some features" popup appeared when you first ran `npx expo start`, click "Allow access".
- As a fallback, run `npx expo start --tunnel` — slower, but routes through the internet instead of the local network, so it works even when Wi-Fi networking is the problem.

### PowerShell says "running scripts is disabled on this system"

This blocks npm's own PowerShell scripts ( `npm.ps1` ), not your code. Fix once, as Administrator:

```
Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
```

### **"npx" is not recognized as an internal or external command"**

Node wasn't installed correctly, or PATH hasn't refreshed. Close every terminal window, reopen PowerShell, and re-run `node -v`. If that still fails, reinstall Node.js.

### **Long path errors on Windows (something about "path too long")**

Windows historically limits file paths to 260 characters, and `node_modules` folders nest deeply. Enable long paths once, as Administrator in PowerShell:

```
New-ItemProperty -Path "HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem" `
  -Name "LongPathsEnabled" -Value 1 -PropertyType DWORD -Force
```

Then restart your PC. Also: keep your project close to the drive root (e.g. `C:\dev\my-app`) rather than deep inside `Documents\Projects\2026\...` — shorter paths mean fewer surprises.

### **Something looks stale / a change isn't showing up**

```
npx expo start -c
```

The `-c` clears Metro's bundler cache — the first thing to try for "I changed the code and nothing happened" type issues.